

Практичне завдання

Побудова Parametric UMAP за допомогою PyTorch

У попередньому практичному занятті ви працювали з реальними даними: досліджували їх, очищували та будували перші візуалізації. Сьогодні ви перейдете до наступного етапу Data Science — **розробки власного алгоритму машинного навчання**.

У цій роботі ви реалізуєте ключові компоненти Parametric UMAP — алгоритму нелінійного зниження розмірності, який дозволяє знаходити компактне двовимірне представлення складних багатовимірних даних.

На відміну від використання готової бібліотеки, цього разу ви самостійно реалізуєте математичні компоненти алгоритму та інтегруєте їх у навчальний пайплайн.

Що таке Parametric UMAP?

UMAP (Uniform Manifold Approximation and Projection) — це алгоритм, який допомагає представляти високовимірні дані у просторі меншої розмірності, зберігаючи їхню структуру.

У цьому практичному завданні ви працюватимете з **Parametric UMAP**, де замість класичного алгоритму використовується нейронна мережа, яка навчається будувати двовимірні представлення зображень MNIST.

Метою є навчити модель так, щоб схожі об'єкти залишалися близькими один до одного після проєкції.

Що ви будете реалізовувати?

Вам потрібно реалізувати шість ключових математичних компонентів Parametric UMAP, які згодом будуть використані у готовому пайплайні навчання моделі.

Функція	Призначення
<code>compute_min_pairwise_distance()</code>	знаходження найближчого сусіда
<code>find_a_b()</code>	підбір параметрів функції схожості
<code>find_sigma()</code>	пошук локального масштабу для кожної точки
<code>compute_asymmetric_p()</code>	побудова асиметричного графа сусідства
<code>compute_symmetric_p()</code>	симетризація графа
<code>umap_loss()</code>	функція втрат, яка навчає модель

Структура проекту

У проєкті вже підготовлено більшу частину коду. Ваше основне завдання — реалізувати алгоритмічні компоненти Parametric UMAP та інтегрувати їх у готову інфраструктуру.

- [student_tasks.py](#)

Основний файл, з яким ви працюватимете під час практичного завдання. Саме тут необхідно реалізувати всі ключові функції Parametric UMAP:

- обчислення відстані до найближчого сусіда;
- підбір параметрів функції схожості;
- пошук локального масштабу ([sigma](#));
- побудову асиметричного та симетричного графів сусідства;
- функцію втрат UMAP.

Кожна функція містить детальний опис математичної постановки задачі, рекомендації щодо реалізації та важливі технічні зауваження.

 Перед початком уважно прочитайте docstring кожної функції — там міститься практично вся необхідна інформація для реалізації.

- [selfcheck.py](#)

Скрипт для самоперевірки реалізації. Він автоматично перевіряє правильність реалізації окремих функцій, допомагає знайти типові помилки та підказує, на якому етапі виникла проблема.

Під час виконання завдання рекомендуємо регулярно запускати:

None

```
python selfcheck.py
```

- [generate_global_probabilities.py](#)

Після реалізації всіх функцій необхідно один раз побудувати глобальний граф сусідства UMAP.

Саме цей скрипт використовує вашу реалізацію для обчислення матриці ймовірностей p_i , яка зберігається на диск і використовується під час навчання моделі.

Запустіть:

None

```
python generate_global_probabilities.py
```

- [boilerplate.py](#)

Містить готовий пайплайн навчання моделі. Після того як:

- реалізовані всі функції у [student_tasks.py](#);
- успішно пройдено [selfcheck.py](#);
- згенеровано глобальний граф за допомогою [generate_global_probabilities.py](#);

можна запускати навчання моделі:

None

```
python boilerplate.py
```

Під час навчання звертайте увагу на значення `val_knn_acc` — саме воно показує, наскільки добре модель навчається зберігати структуру даних після зниження розмірності.

Підготовка середовища

Перед початком роботи переконайтеся, що у вас встановлено:

- Python 3.11+
- PyTorch
- Torchvision
- Lightning
- NumPy
- SciPy
- Scikit-learn

Орієнтовні версії пакетів, з якими тестувалося завдання:

None

```
lightning==2.6.5
```

```
numpy==2.5.1
```

```
scikit-learn==1.9.0
```

```
scipy==1.18.0
```

```
torch==2.13.0
```

```
torchvision==0.28.0
```

💡 Не обов'язково встановлювати пакети саме цих версій. Невеликі відмінності (наприклад, інша мінорна або патч-версія) зазвичай не впливають на виконання завдання.

Водночас рекомендуємо використовувати сучасні версії бібліотек — якщо ваші пакети суттєво застарілі, можуть виникнути проблеми із сумісністю або запуском коду.

Частина 1: Ознайомлення з кодовою базою

Перед початком роботи:

- відкрийте всі чотири файли;
- перегляньте структуру проєкту;
- знайдіть місця, які потрібно реалізувати;
- зверніть увагу на TODO-блоки.

💡 Не поспішайте одразу писати код. Спочатку спробуйте зрозуміти загальну логіку роботи проєкту.

Частина 2: Реалізація функцій

Послідовно реалізуйте всі функції у `student_tasks.py`.

Після реалізації кожної функції рекомендується одразу перевірити її роботу.

💡 Не відкладайте перевірку "на потім". Знайти одну помилку після написання 300 рядків коду набагато складніше.

Частина 3: Самоперевірка

Після реалізації всіх функцій виконайте:

None

```
python selfcheck.py
```

Якщо тест повідомляє про помилку:

- уважно прочитайте текст помилки;
- поверніться до відповідної функції;
- звірте реалізацію з описом у docstring.

Частина 4: Побудова глобального графа

Після того як усі функції реалізовані та `selfcheck.py` проходить без помилок,

необхідно побудувати глобальний граф сусідства.

Для цього виконайте:

None

```
python generate_global_probabilities.py
```

Цей крок потрібно виконати лише один раз. Він використовує вашу реалізацію для побудови графа, який надалі використовуватиметься під час навчання моделі.

Частина 5: Навчання моделі

Коли всі перевірки успішно проходять:

None

```
python boilerplate.py
```

Під час навчання звертайте увагу на:

- втрати (loss);
- значення `val_knn_acc`;
- стабільність навчання.

Що потрібно здати?

Після завершення практичної роботи:

1. Заархівуйте **всю робочу директорію проекту**.
2. **Не додавайте до архіву директорію `data/`**, оскільки вона містить вхідні дані й значно збільшує розмір архіву.
3. Завантажте отриманий **.zip**-архів через форму задачі домашнього завдання.



Перед архівацією перевірте, що у проєкті залишилися всі змінені файли, зокрема `student_tasks.py` та інші файли, які ви редагували під час виконання

завдання.

✓ Перед здачею переконайтеся, що:

- `selfcheck.py` проходить без помилок;
- проєкт успішно запускається;
- усі реалізовані функції збережені;
- директорія `data/` **не входить** до архіву;
- ви завантажуєте саме `.zip`-архів із робочою директорією.